- Origin [CORS flow depends on the value of this request header]

The CORS response headers are as follows:

- Access-Control-Allow-Headers
- Access-Control-Allow-Methods
- Access-Control-Allow-Origin
- Access-Control-Allow-Credentials

The browser checks for the CORS response headers and decides whether to make the actual request or not.

Figure 2 is a quick reference for the conditions in which CSRF is possible.

CSRF guarding mechanisms

The effects of CSRF cannot be underestimated and defence against it should be well thought of. So let's discuss the common defence mechanisms to protect against CSRF:

- Use of CSRF tokens
- Use of SameSite attribute
- CORS
- Use of fetch meta headers

Use of CSRF tokens: This is a very commonly used method for CSRF protection. The application owner must first identify the sensitive requests which need CSRF protection. Normally, all the state changing methods are sensitive requests. In this case, the application server generates the dynamic token (random value which is not easily guessable) and the browser includes this token value in the specific request header (custom). This request header is a custom header. The value of such a header is set with the value of the token generated and the request is sent. The application server, upon receiving the request, checks and validates the header and its value. If the validation is successful, it processes the request and responds.

You can refer to the sequence diagram in Figure 3.

Use of SameSite attribute: We have already discussed how CSRF protection can be enforced using the SameSite attribute. Developers can refer to meta headers (fetch specifications) set by the browser implementations for a quick reference on which requests the browser is treating as same-site and cross-site.

- *Samesite=strict*

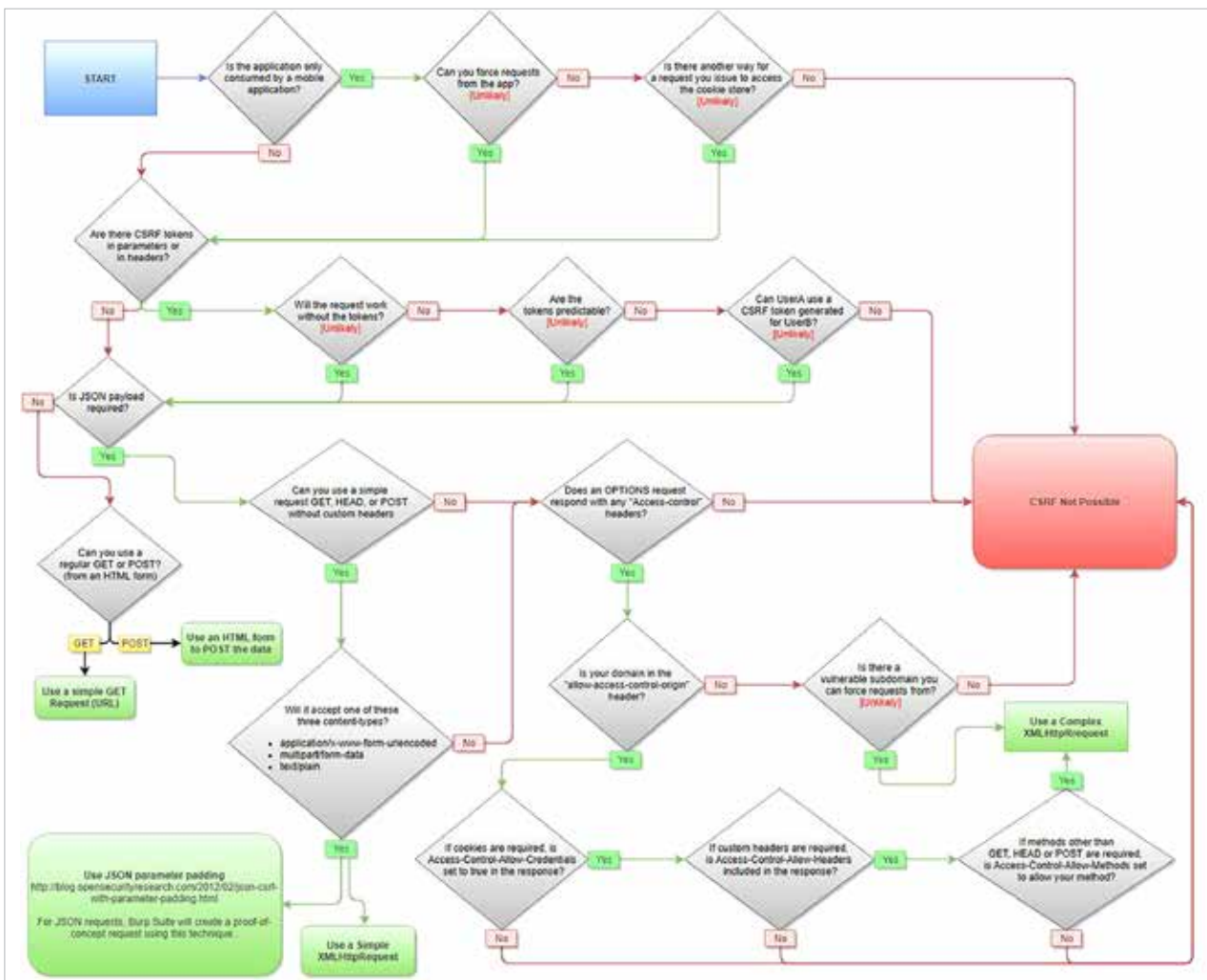


Figure 2: CSRF map